

PRD + Tech Plan: سیستم درخواست کالا/خرید (MVP)

هدف و دامنه (1)

- پیاده‌سازی دمو بک‌اند و سپس فرانت برای جریان‌های «درخواست کالا از انبار» و «درخواست خرید» با هدف: نقش‌های جداگانه.
- نقش‌ها:
 - (درخواست‌کننده از انبار) Requester
 - (انباردار) Warehouse Manager
- کاربرها از قبل در دیتابیس مقداردهی می‌شوند؛ احراز هویت فقط با اطلاعات از پیش موجود: **عدم ثبت نام آزاد** و انتخاب نقش درست.
- در صورت خطا Auth لینک/بخش تماس با پشتیبانی در صفحه: **پشتیبانی**.

نقشه مسیر کاربر (User Journeys) (2)

شناسه کاربر (مثلاً + role: ورودی‌ها - Auth. 2) → ورود به داشبورد → صفحه انتخاب نقش «CTA - Landing 1) اگر نقش انتخابی با نقش ذخیره‌شده کاربر - (ساده passcode/OTP برای MVP: passcode) + (ایمیل/موبایل/کدملی فرم «درخواست کالا از انبار» + لیست - Requester Dashboard (هم‌خوانی نداشت → خطا + لینک پشتیبانی. 3 «تب» «فرم‌های درخواست کالا از انبار - Warehouse Manager Dashboard (درخواست‌های ارسال شده من». 4 «مشاهده/تایید/رد/تبدیل به «درخواست خرید». (بدون امکان ویرایش فرم درخواست کالا) - تب «فرم‌های درخواست خرید با فشار دکمه: **تبدیل** - امکان ایجاد/ویرایش فقط برای انباردار. (Draft/Submitted/Approved/Rejected) لیست فرم خرید تا حد ممکن خودکار پر می‌شود، Convert.

حالت‌ها و قوانین (3)

درخواست کالا از انبار (WarehouseRequest) (3.1)

- **Status:** SUBMITTED | APPROVED | REJECTED | CONVERTED
- است (قابل ویرایش نیست) **immutable** درخواست، SUBMITTED بعد از.

درخواست خرید (PurchaseRequest) (3.2)

- **Status:** DRAFT | SUBMITTED | APPROVED | REJECTED
- فقط انباردار می‌تواند ایجاد/ویرایش کند.
- مرتبط دارد (قیود یکتا) PurchaseRequest حداکثر یک WarehouseRequest هر.

قواعد دسترسی (RBAC) (3.3)

عمل	Requester	Warehouse Manager
ورود (Auth)	REQUESTER فقط اگر نقش‌اش	MANAGER فقط اگر نقش‌اش
ایجاد WarehouseRequest	✓	✗
خود WarehouseRequest مشاهده	✓	✗

عمل	Requester	Warehouse Manager
WarehouseRequest مشاهده همه	✗	✓ (فقط خواندنی)
WarehouseRequest تایید/رد	✗	✓
PurchaseRequest تبدیل به	✗	✓
PurchaseRequest ایجاد/ویرایش	✗	✓

4) توصیفی (ERD) مدل داده

- **User**(id, email?, phone?, nationalId?, role[REQUESTER|MANAGER], passcode (MVP), isActive)
- **WarehouseRequest**(id, requesterId→User, title, description?, items[JSON], requiredDate?, status, lockedAt?, createdAt, **itemCode**, **specs**, **usageLocation**, **priority**)
- **PurchaseRequest**(id, sourceWRequestId→WarehouseRequest?, managerId→User, items[JSON], vendor?, total?, status, createdAt, **usageLocation**, **priority**)
- **AuditLog**(id, actorId→User, entityType, entityId, action, payload[JSON], at)
- **status/createdAt**: ایندکس‌ها روی FK: UNIQUE(PurchaseRequest.sourceWRequestId): قیود

NEW Database Schema) بهبود ا: تکمیل مدل داده

- افزودن فیلدهای کلیدی به جداول اصلی: `itemCode`, `specs`, `usageLocation`, `priority`, `description`.
- مزیت: از دست رفتن اطلاعات جلوگیری می‌شود و ساختار دیتا با فرم‌های واقعی همخوان است.

5) خلاصه API (Contract)

- **Auth**
 - `POST /auth/login` {role, identifier, passcode} → {accessToken, user}
 - پشتیبانی link + سناریوی خطا: 401 (نقش اشتباه یا کاربر غیرفعال).
- **Requester**
 - `POST /requests` (auth: REQUESTER) → ایجاد WarehouseRequest
 - `GET /requests/my` → لیست درخواست‌های خود کاربر
- **Manager**
 - `GET /manager/requests` → WarehouseRequest لیست همه با status
 - `POST /manager/requests/:id/approve` | `.../reject`
 - `POST /manager/requests/:id/convert` → مرتبط PurchaseRequest ایجاد/برگرداندن
 - `GET /manager/purchases` | `GET /manager/purchases/:id`
 - `POST /manager/purchases` (ایجاد مستقیم)

5.1) JSON نمونه اسکیمای

```
// WarehouseRequest.create
{
  "title": "A4 درخواست تامین",
  "description": "مصرف ماه جاری",
}
```

```

"itemCode": "A4-80G",
"specs": "گرم، 500 برگ 80",
"usageLocation": "دفتر مرکزی",
"priority": "HIGH",
"items": [
  {"sku": "A4-80G", "name": "کاغذ A4", "qty": 10, "unit": "بسته"}
],
"requiredDate": "2025-10-01"
}

```

```

// convert → PurchaseRequest (خروجی)
{
  "id": "pr_123",
  "sourceWRequestId": "wr_456",
  "items": [
    {"sku": "A4-80G", "name": "کاغذ A4", "qty": 10, "unit": "بسته"}
  ],
  "usageLocation": "دفتر مرکزی",
  "priority": "HIGH",
  "status": "DRAFT"
}

```

NEW **(Convert) بهبود ۲: بهبود منطق تبدیل**

- بین فیلدهای mapping طراحی WarehouseRequest و PurchaseRequest.
- انتقال خودکار داده‌های کلیدی (itemCode, specs, description, usageLocation, priority).
- توسط انباردار تکمیل شود (vendor, total) تنها داده‌های جدید.

NEW **API Endpoints بهبود ۳: تکمیل**

- های جزئیات endpoint افزودن:
- GET /requests/:id → شامل همه فیلدهای جدید WR نمایش کامل.
- GET /purchases/:id → PR نمایش کامل.
- POST /manager/requests/:id/convert → پشتیبانی از دریافت مقادیر جدید (usageLocation, priority) هنگام تبدیل.
- مزیت: پشتیبانی از تبدیل دقیق‌تر، واکنشی کامل دیتا و توسعه‌پذیری بالا.

(Acceptance) پذیرش و تست‌ها 6)

- **Auth**
- اگر نقش انتخابی با نقش واقعی کاربر هم‌خوان نیست → 401 + پیام «نقش انتخابی صحیح نیست» + لینک پشتیبانی.
- **Requester**
- و پس از ایجاد، غیرقابل ویرایش status=SUBMITTED → ایجاد درخواست موفق.

- `GET /requests/my` لیست همه درخواست‌ها را برمی‌گرداند.
- **Manager**
 - WarehouseRequest (خواندنی) مشاهده همه.
 - Audit می‌شود وضعیت را تغییر می‌دهد و Approve/Reject.
 - Convert (idempotent) جدید می‌سازد یا قبلی را برمی‌گرداند PurchaseRequest یک
- **Consistency**
 - قابل مشاهده/تبدیل refresh در تب «فرم درخواست خرید» پس از Requester، بعد از ارسال درخواست توسط.

7) Roadmap (ترتیب پیاده‌سازی)

Users, VS Code و کد نویسی در Supabase (PostgreSQL + Auth + API) با **Backend Scaffold** 1)
Landing + (اولیه: 1 مدیر + چند درخواست کننده نمونه. 2. Seeder - WarehouseRequests, PurchaseRequests, Audit.
 + فرم ایجاد درخواست - **Requester UI** (خطا/موفق + پشتیبانی. 3 → Auth انتخاب نقش → فرم - **Auth** (فرانت ساده)
 لیست - Approve/Reject/Convert. لیست درخواست های انبار + عمل های - **Manager UI** (لیست ارسال ها. 4
 SUBMITTED, Audit. پس از Request قید یکتایی، تبدیل، قفل ویرایش - **اتصالات و قیود** (درخواست های خرید. 5

8) MVP (پیشنه‌گذاری Stack)

- **Backend:** Supabase (Postgres, Auth, Row Level Security), Edge Functions لايچيڪھا
- **Frontend:** React (Vite) يا Next.js؛ Zustand/React Query؛ Tailwind.
- **IDE:** Visual Studio Code (VS Code).
- **DevOps:** Supabase hosting + GitHub Actions (lint/test).

9) پیشنهادی Supabase اسکیمای

```
alter table warehouse_requests add column item_code text;
alter table warehouse_requests add column specs text;
alter table warehouse_requests add column usage_location text;
alter table warehouse_requests add column priority text;

alter table purchase_requests add column usage_location text;
alter table purchase_requests add column priority text;
```

10) Endpoint جزئیات (Edge Functions در Supabase)

- **Auth**
- POST /auth/login → mismatch نقش را با کاربر چک می‌کند؛ روی 401.
- **Requests**
- POST /requests → Patch ست شده و دیگر locked_at، پس از ایجاد WR ایجاد
- GET /requests/my
- GET /requests/:id
- **Manager**
- GET /manager/requests?status=

- `POST /manager/requests/:id/approve|reject`
- `POST /manager/requests/:id/convert` (جدید usageLocation, priority دریافت)
- `GET /manager/purchases`
- `GET /purchases/:id`
- `POST /manager/purchases`

11) قواعد مهم کسب و کار

- را ویرایش کند WarehouseRequest نمی تواند هرگز انباردار.
- ساخته شده بود، همان را برگرداند PR باشد: اگر قبلاً **idempotent** باید (Convert) تبدیل.
- ذخیره شود AuditLog هر تغییر وضعیت در.

12) UI Routes (بعد از بک اند)

- `/` Landing (CTA: ورود به داشبورد)
- `/auth?role=requester|manager`
- `/dashboard/requester` → (Create Request + My Requests)
- `/dashboard/manager` → Tabs: (Warehouse Requests) | (Purchase Requests)
- `/support` (اختیاری)

13) پذیرش نهایی دمو

- ببیند/تایید کند → تبدیل به Manager → وارد شود → ارسال فرم Requester end-to-end: سناریو
- مشاهده در تب مربوطه PurchaseRequest →

14) چک لیست تست دستی

- خطا + لینک پشتیبانی → Auth نقش غلط در.
- دسترسی پیدا کند (403) و برعکس Manager نتواند به مسیرهای Requester.
- قابل مشاهده/تبدیل باشد refresh پس از Purchase در تب WR، پس از ارسال.
- (idempotency) جدید نسازد PR، تبدیل دوم.

15) نکات فرانت/طراحی (پس از بک اند)

- استفاده از پالت رنگی داده شده.
- CTA با `--pumpkin` / `--sunshade` پس زمینه با `--quarter-spanish-white`.
- Micro-interactions شیشه ای (Glass) سبک؛ header/modal فقط برای.

Next: طبق Auth + Users + Requests + Purchases و پیاده سازی Supabase با (Backend Scaffold) شروع مرحله 1 این سند.